



UTEC Posgrado

CURSO

INGENIERÍA DE DATOS

Angel Tintaya, Tech MBA
Senior Data Engineer



The background of the image features a dark blue field with a glowing network of white lines and dots, resembling a data or neural network. In the lower foreground, a human hand is visible, with the index finger pointing towards the left, where a bright light source creates a lens flare effect. The text is centered in the upper half of the image.

ARQUITECTURA Y ESTRATEGIA DE DATA ENGINEERING



PATRONES DE INGESTIÓN DE DATOS

¿ Cuánto tiempo puede esperar una empresa antes de reaccionar a sus datos?



Qué aprenderemos hoy

- Qué son los patrones de ingestión de datos
- Diferencias entre Batch y Streaming
- Cómo funcionan Lambda y Kappa
- Cuándo utilizar cada enfoque
- Un caso práctico basado en Machine Learning y eventos



AGENDA DE HOY

- The road so far
- ¿Qué son los patrones de ingestión de datos?
- Batch vs Streaming
- Lambda: combinando Batch y Streaming
- Kappa: un único flujo de eventos
- Caso práctico: Machine Learning sobre eventos





The road so far



- Fundamentos de Data Engineering
- Diferencias entre Data Engineering, Data Discovery y Data Science
- Arquitecturas modernas de datos: Data Warehouses, Data Lakes y Lakehouse
- Data Mesh y Data Fabric: enfoques modernos para la gestión de datos
- **Patrones de ingestión de datos: Batch, Streaming y Lambda/Kappa Architectures**



UTEC VENTURES

Patrones de Ingestión

PANEL UV

UIS 20

UTEC

repositorio

¿Qué son los patrones de ingestión de datos?

Los patrones de ingestión definen cómo los datos ingresan y son procesados dentro de una plataforma de datos.

La elección del patrón impacta directamente:

UBER

Procesos
Streaming



Procesos
Batch



¿Qué son los patrones de ingestión de datos?

Los patrones de ingestión definen cómo los datos ingresan y son procesados dentro de una plataforma de datos.

La elección del patrón impacta directamente:

Latencia

Costos

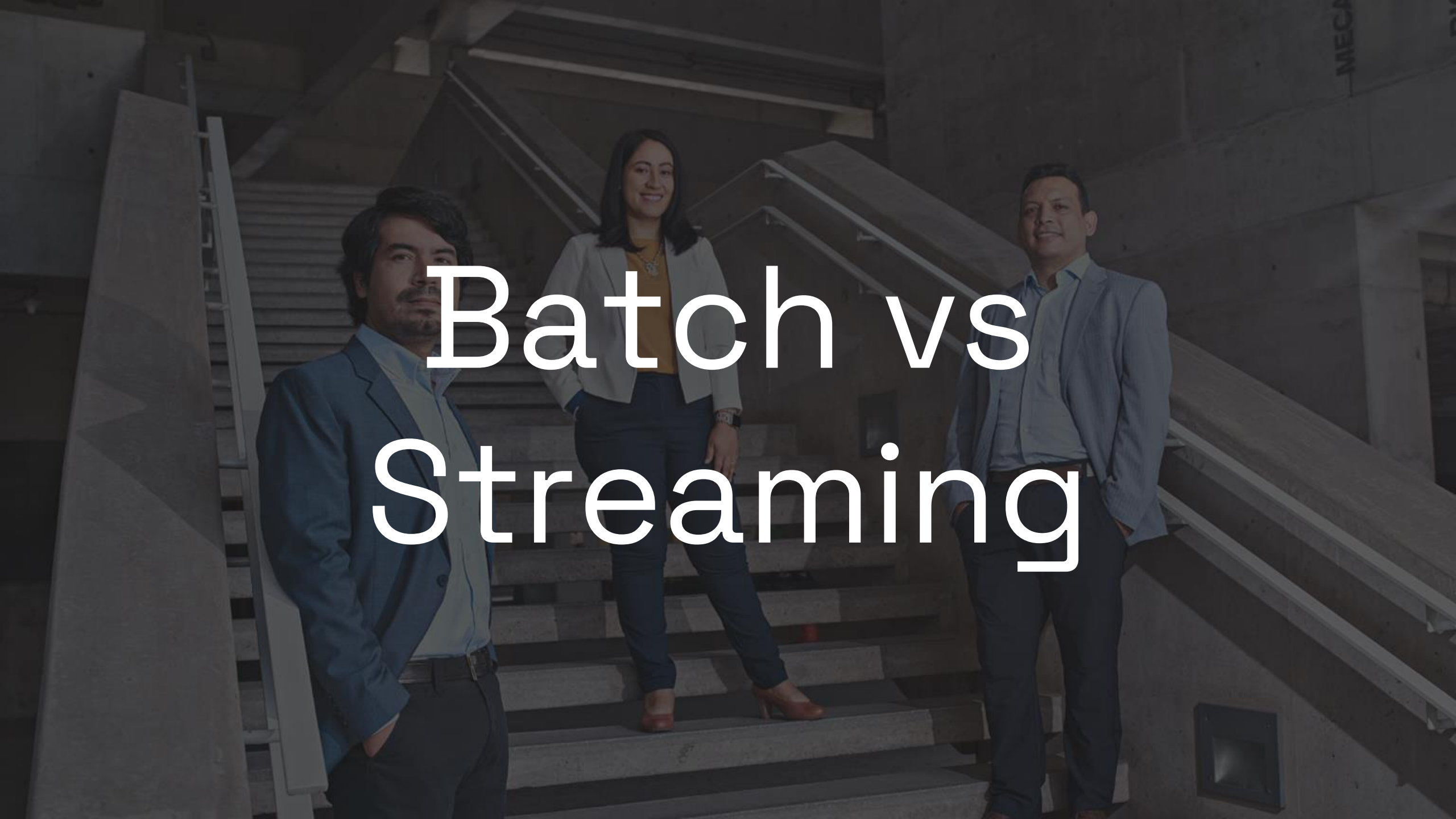
**Complejidad
Operativa**

**Capacidad de
Respuesta**



¿ Todos los problemas requieren la misma velocidad de respuesta?



A photograph of three people—two men and one woman—standing on a wide, modern concrete staircase. The man on the left is wearing a blue blazer over a light blue shirt and dark trousers. The woman in the center is wearing a light grey blazer over a yellow top and dark trousers. The man on the right is wearing a light blue blazer over a light blue shirt and dark trousers. They are all smiling and looking towards the camera. The background is a modern building with concrete walls and stairs. The text "Batch vs Streaming" is overlaid in the center in a large, white, sans-serif font.

Batch vs Streaming

A hand is shown in the lower foreground, pointing its index finger towards a glowing network diagram on the left. The diagram consists of white nodes connected by lines, set against a dark blue background with a bright light source creating a lens flare effect. The text is centered in the middle of the image.

Si una empresa puede esperar horas o días para analizar datos... ¡no necesita realtime!

¿Qué procesos de tu empresa podrían ejecutarse una vez al día sin problema?



¿Qué es Ingesta Batch?

Definición:

Procesamiento de grandes volúmenes de datos acumulados a intervalos regulares.

Características:

Alta eficiencia para grandes volúmenes

Facilidad para reintentar y auditar

Latencia temporal (ej. diario, semanal)

Ejemplos:

Procesos programados (Airflow + Python + Azure Datalake)

Entrenamiento mensual de modelos



Ejemplo: Ingesta Batch en Airflow

Dag

☒ 02_my_dag_normal

Dag Run

2025-05-08, 19:00:00

Options ▾

training_model_2

@Task

✓ success

training_model_1

@Task

✓ success

training_model

@Task

✓ success

choose_best_model

@Task.Branch

✓ success

accurate

@Task

❌ skipped

inaccurate

@Task

✓ success

2025-05-08, 19:00:00 ✓ success

Add a note Clear Run Mark Run as... ▾

Logical Date	Run Type	Start	End
2025-05-08, 19:00:00	🕒 scheduled	2025-05-08, 20:13:44	2025-05-08, 20:13:44

Duration

74.54s

Task Instances Events Code Details

```

20 # 2. Define the DAG
21 @dag(
22     dag_id="02_my_dag_normal",
23     default_args=default_args,
24     schedule=MultipleCronTriggerTimetable("@daily", timezone="UTC"),
25     tags=["example", "training"],
26 )
27 def my_training_pipeline():

```

¿Qué pasa si esperar horas genera pérdidas?



Sistemas que fallarían si reaccionaran tarde:



Fraude



IoT



Recomendaciones



Monitoreo



¿Qué es Ingesta Streaming?

Definición:

Procesamiento continuo de datos a medida que llegan
(event-by-event o micro-batch).

Características:

Baja latencia

Procesamiento en tiempo real o casi real

Mayor complejidad (estado, orden, fallos)

Ejemplos:

Procesamiento de logs o sensores (Spark Streaming)

Recomendaciones personalizadas en tiempo real



Ejemplo: Ingesta Streaming en Databricks

Workflows > Jobs >

job_basic_streaming ☆

Runs Tasks

Runs

Start date  < Previous Next >

Run total duration

3m 38s

1m 49s

May 08

stream_to_table

Tasks

Cancel runs ▾

Start time	Run ID	Launc...	Duration	Spark	Status	Error code	Run para...
May 08, 202...	10317...	Manually	3m 44s	Spark UI / Logs / Metrics	 Running		

```
# Ruta de origen (raw files)
source_path = "abfss://datalake@stdemdsai.dfs.core.windows.net/raw/clientes/"

# Nombre de la tabla destino (Unity Catalog)
table_name = "my_catalog.my_schema.clientes_csv"

# Leer stream desde archivos CSV
df_stream = (
    spark.readStream
        .format("csv")
        .option("header", "true")
        .schema(schema)
        .load(source_path)
)

# (Opcional) Agregar columna de trazabilidad
df_stream = df_stream.withColumn("source_file", input_file_name())

# Escribir como tabla Delta gestionada por Unity Catalog
query = (
    df_stream.writeStream
        .format("delta")
        .option("checkpointLocation", "abfss://datalake@stdemdsai.dfs.core.windows.net/_checkpoints/clientes_csv")
        .outputMode("append")
        .toTable(table_name)
)

```

¿Netflix necesita batch o streaming?



¿Todo Netflix necesita realtime?



Batch vs Streaming

Característica	Ingesta Batch	Ingesta Streaming
Latencia	Alta (minutos, horas)	Baja (segundos o menos)
Complejidad	Baja	Alta
Tolerancia a fallos	Simple (retry)	Compleja (Checkpoint, estado)
Casos de uso	Reportes, análisis	Alertas, monitoreo, IoT
Herramientas típicas	Airflow, Pandas	Spark Streaming, Kafka

Batch vs Streaming – Herramientas Clave

Categoría	Batch	Streaming
Motores	Apache Spark, dbt, Databricks (con Spark)	Spark Structured Streaming, Apache Flink, Databricks, Kafka
Orquestadores	Airflow, Azure Data Factory, AWS Glue	Airflow (lanzando notebooks/scripts), Delta Live Tables (DLT)



Lambda Architecture

**¿Qué pasa si necesito velocidad... pero
también precisión histórica?**



¿Qué es Lambda Architecture?

Lambda Architecture es una arquitectura de procesamiento de datos que **combina** un flujo de **datos en tiempo real** (streaming) con un **procesamiento por lotes** (batch) para **lograr** tanto **baja latencia** como **precisión histórica** en grandes volúmenes de datos.



Las tres capas de Lambda Architecture

Batch Layer

Procesamiento completo, preciso,
pero con latencia.

Speed Layer

Procesamiento sobre datos recientes,
sin esperar a que se procese en batch

Serving Layer

Expone resultados
desde ambas capas
para ser consumidos.



Lambda Architecture – Caso de Uso

Vista de historias de una red social



La vista histórica se procesa en batch,
y se actualiza rápidamente con
eventos nuevos en streaming.



Lambda Architecture – Ventajas y Desventajas

Ventajas

- Combina precisión (batch) y velocidad (streaming).
- Resiliente: El batch puede corregir errores del streaming.

Desventajas

- **Mucha complejidad:** Necesidad de mantener dos sistemas distintos (Código duplicado, sincronización)
- Dificultad operativa.

Lambda Architecture – Ejemplo Práctico

```
%sql
SELECT * FROM my_catalog.my_schema.all_tweets
```

► (2) Spark Jobs

► `_sqldf: pyspark.sql.dataframe.DataFrame = [created_at: timestamp, current_comme`

Table ▾ +

	 created_at	^A _C current_comment	^A _C source
1	2025-05-01T00:00:00.000+00:00	First comment	batch
2	2025-05-02T00:00:00.000+00:00	Second comment	batch
3	2025-05-03T00:00:00.000+00:00	Third comment	batch
4	2025-05-04T00:00:00.000+00:00	Fourth comment	batch
5	2025-05-05T00:00:00.000+00:00	Fifth comment	stream
6	2025-05-06T00:00:00.000+00:00	6th comment	stream
7	2025-05-07T00:00:00.000+00:00	7th comment	stream
8	2025-05-08T00:00:00.000+00:00	8th comment	stream
9	2025-05-09T00:00:00.000+00:00	9th comment	stream

Batch Layer:

- Tweets hasta el día anterior

Speed Layer:

- Tweets actuales en realtime.

Serving Layer:

- Todos los tweets consolidados

Nota: Lambda Architecture no significa una tabla única, sino que ambos flujos (batch y stream) existen en paralelo y sus resultados pueden unirse o no, dependiendo del caso de uso.



A background image showing two women in an office. The woman on the left is smiling broadly, looking towards the right. The woman on the right is also smiling, looking at a computer screen and pointing at it with her right hand. The image is dimmed to serve as a background for the text.

Kappa Architecture

¿Y si pudiéramos simplificar Lambda y procesar todo desde un mismo stream?



¿Qué es Kappa Architecture?

Es una propuesta de Jay Kreps (cofundador de Apache Kafka), se propone hacer todo en streaming, incluso lo que antes se hacía como batch.

Si necesitas reprocesar, simplemente vuelves a reproducir el stream desde el inicio (por ejemplo, desde Kafka).



Kappa Architecture – Ventajas y Desventajas

Ventajas

- Simplicidad de mantenimiento (una sola vía de procesamiento).
- Se adapta bien a sistemas modernos con alto volumen de eventos.

Desventajas

- Reprocesar desde el inicio puede ser costoso si el volumen es muy grande.
- No siempre es fácil hacer todo en streaming (hay algoritmos que aún requieren batch).

Kappa Architecture – Ejemplo Práctico

```
%sql
SELECT * FROM my_catalog.my_schema.all_tweets_kappa
```

► (1) Spark Jobs

► `_sqldf: pyspark.sql.dataframe.DataFrame = [created_at: timestamp, current_comme`

Table ▾ +

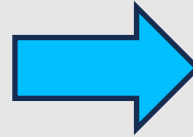
	 created_at	^A _C current_comment	^A _C source
1	2025-05-01T00:00:00.000+00:00	First comment	stream
2	2025-05-02T00:00:00.000+00:00	Second comment	stream
3	2025-05-03T00:00:00.000+00:00	Third comment	stream
4	2025-05-04T00:00:00.000+00:00	Fourth comment	stream
5	2025-05-05T00:00:00.000+00:00	Fifth comment	stream
6	2025-05-06T00:00:00.000+00:00	6th comment	stream
7	2025-05-07T00:00:00.000+00:00	7th comment	stream
8	2025-05-08T00:00:00.000+00:00	8th comment	stream
9	2025-05-09T00:00:00.000+00:00	9th comment	stream

Todo en un único stream



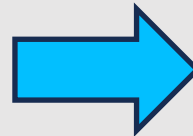
¿Cuándo es mejor usar Kappa en lugar de Lambda?

Necesitas un sistema más simple



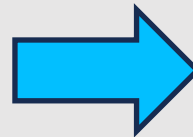
No duplicas lógica en batch + streaming. Un solo pipeline.

Datos llegan continuamente y en alto volumen



Kappa está diseñado para flujos constantes y persistentes (ej: logs, eventos IoT).

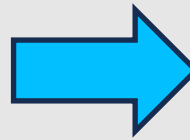
Latencia importa mucho



Kappa es naturalmente en tiempo real, sin esperar al batch.

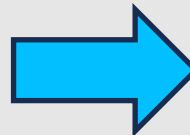
¿Cuándo es mejor usar Kappa en lugar de Lambda?

Reprocesamiento es raro o factible



Si usas Kafka o Spark Streaming, puedes simplemente volver a consumir desde el inicio.

Modelos ML online o reglas complejas sobre eventos



Kappa favorece el stream como primera clase, permitiendo aplicar lógica avanzada directamente.



¿Cuándo NO usar Kappa?

- Cuando necesitas **reprocesar terabytes de datos** con lógica pesada.
- Cuando tu **fuentes no es un stream real**, sino datos que se actualizan de forma discontinua (como hojas de cálculo).
- Si **no tienes infraestructura robusta** para manejar logs de eventos duraderos (como Apache Kafka o Delta Lake en Databricks con Structured Streaming)



Bash vs Streaming vs Lambda vs Kappa

Concepto	Tipo	Procesamiento	Complejidad	Latencia
Batch	Modo de procesamiento	Lotes	Baja	Alta
Streaming	Modo de procesamiento	En tiempo real	Media-Alta	Baja
Lambda	Arquitectura	Batch + Streaming	Alta	Mixta
Kappa	Arquitectura	Sólo Streaming	Media	Baja

¿Cuándo usar cada concepto?

Concepto	¿Cuándo usar?	Ejemplo	Ventajas	Limitaciones
Batch	Datos acumulados, baja latencia no es crítica	Reporte mensual de ventas, entrenamiento de modelo de predicción cada semana	Preciso, fácil de depurar	No apto para casos en tiempo real
Streaming	Necesitas resultados en tiempo real o casi	Detección de fraude, alertas en IoT, dashboard de actividad en vivo	Respuesta rápida, datos actualizados	Complejo de mantener, precisión limitada si no hay backfill
Lambda	Quieres precisión batch pero también velocidad	Análisis financiero con vista histórica y alertas en vivo	Balance entre exactitud y velocidad	Duplicación de lógica y arquitectura
Kappa	Sistema solo basado en eventos, procesamiento unificado, menor complejidad	Monitor de tráfico, recomendaciones en tiempo real, modelos online con reentrenamiento	Un solo código para todo, más simple que Lambda	Reentrenamiento sobre streams no siempre es suficiente, no todos los algoritmos aplican

Caso práctico: Machine Learning sobre eventos

The background image shows a modern office space with large glass windows and doors. The text is overlaid on this image. The office interior is visible through the glass, showing desks, chairs, and various office equipment. The text is in a large, white, sans-serif font, centered on the image. The overall tone is professional and modern.

¿Qué pasa si los patrones cambian más rápido de lo que tu modelo puede aprender?



¿Por qué Kappa es ideal para modelos de ML online?

Un **modelo ML online** (o de aprendizaje continuo) **se entrena o actualiza en tiempo real**, a medida que llegan nuevos datos. No espera a tener todo el dataset junto como en el batch.

Necesita:

- **Eventos recientes** (para detectar cambios de comportamiento, "concept drift").
- **Flujo constante de datos** (inputs que llegan sin parar).
- **Latencia baja** (para reaccionar rápido, por ejemplo, a un fraude).



En Kappa

- El sistema **nativamente está basado en eventos**.
- Puedes tener un proceso que consume los datos, **actualiza un modelo incremental** y sirve predicciones casi en tiempo real.
- No necesitas reentrenar desde cero cada cierto tiempo (como sí pasaría en Lambda con batch retraining).

Ejemplo:

Un sistema de detección de fraude bancario que se adapta en tiempo real a nuevas técnicas de fraude.



... ¿Es suficiente usar sólo modelos incrementales?

- Los modelos incrementales suelen hacer **supuestos fuertes** (como distribución estable o cambios graduales), que no siempre se cumplen.
- Si el modelo empieza a cometer **errores por "concept drift"**, puede **necesitar ser reseteado** y reentrenado desde cero.



Recomendación:

Combinación de enfoque streaming y batch:

- Un **modelo incremental que se actualiza en tiempo real** (útil para decisiones inmediatas).
- Y cada cierto tiempo (ej: una vez al día o por semana), se hace un **reentrenamiento batch completo**, incorporando todos los datos históricos y las nuevas observaciones.



Modelo Incremental + reentrenamiento batch = Kappa?

Si, cuando:

- Sólo hay un flujo de datos en streaming.
- El modelo se entrena online con cada evento.
- Cada cierto tiempo, se reentrena el modelo completo usando los eventos almacenados en ese mismo stream.
- No hay pipelines separados, todo parte de la misma fuente.



Kappa no significa que todo sea en tiempo real, sino que todo parte de un mismo stream. Si tus modelos se entrenan (o reentrenan) desde ese flujo de eventos, y no tienes dos caminos paralelos batch/stream separados, entonces estás en Kappa, aunque uses procesamiento batch de vez en cuando.





Takeaways

- No todos los sistemas necesitan tiempo real.
- Batch y Streaming responden a necesidades distintas de negocio.
- Lambda combina velocidad y precisión, pero aumenta complejidad.
- Kappa simplifica el procesamiento usando un único flujo de eventos.
- Los modelos de ML online requieren datos continuos y baja latencia.
- La mejor arquitectura depende del caso de uso y capacidad operativa.

¡Gracias!



UTEC Posgrado